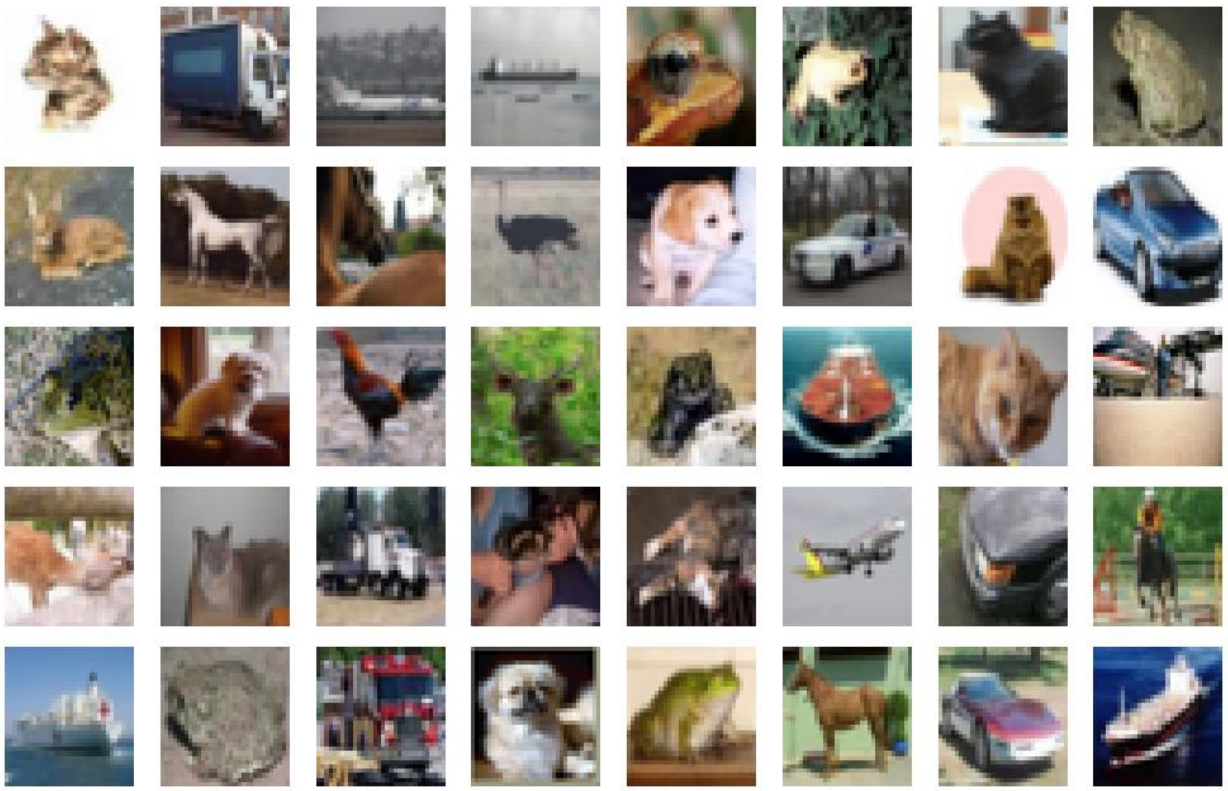
Improving on CIFAR10 dataset using Convolution Neural Network

by: Deelip Solanki



Content

1. A brief overview of the project
2. Result Summary Table
3. Starting with a simple CNN model
4. Regularization to minimize overfitting
5. Modifying the architecture
6. Hyperparameter tuning
7. Data augmentation and a Deep CNN model
8. Transfer learning with ResNet
9. Error Analysis
10. Generating predictions for my images
11. Acknowledgement

A Brief Overview of Project

The dataset contains **32 x 32 RGB images**. Each image is labeled into one of 10 distinct categories:

airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck

Our goal is to train a model that can learn to classify an image to its correct label. This is a supervised learning problem and the original CIFAR-10 dataset contains 60,000 labelled images. But we will be using **50,000 images** (available in csv format on Kaggle) for training and evaluating our model.

The data is then further split into two sets:

- **40,000** for training
- **10,000** for evaluating

We will train **Convolutional Neural Network** model for this project.

Unlike traditional ML algorithms, CNNs can automatically extract features from images..

We'll start with a simple CNN model and keep modifying it further by introducing early stopping, regularization, data augmentation etc. to improve the results. We will try changing the architecture of our model and in the end, we'll use a pretrained model (ResNet18- pretrained on ImageNet dataset of 1 million images) to achieve best results.

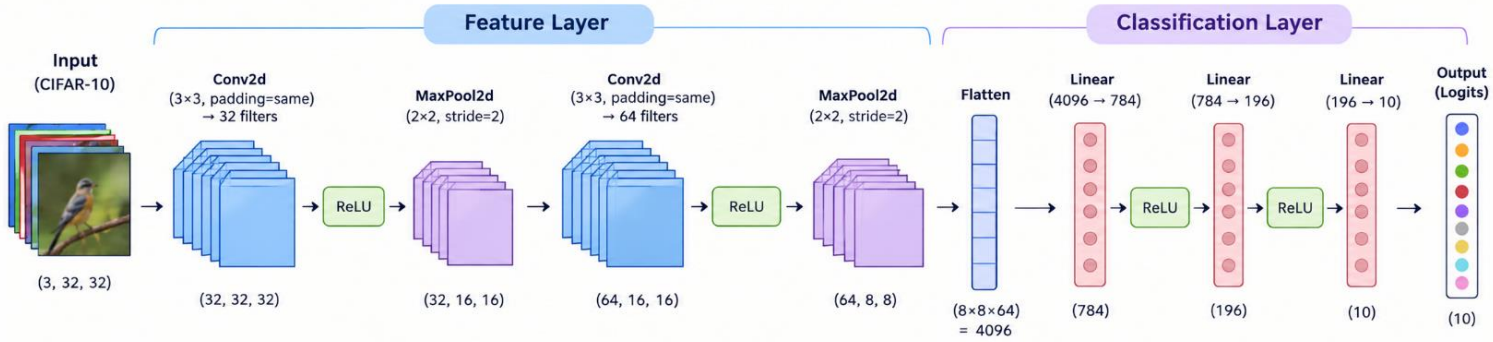
The classes are all balanced so we can use **accuracy** as a reliable metric throughout to track the model's performance.

Result Summary Table

Before you go ahead, take a look at this table which summarizes the results of different models.

Model	Variant	Data Augmentation	Training accuracy (%)	Validation Accuracy (%)	Performance change
Base CNN	-	no	97.63	69.52	
Base CNN	with regularization	no	90.45	73.89	↑
	Pooling removed	no	86.97	64.08	↓
	Reduced classification layer	no	91.48	75.16	↑
	Increased convolution layers	no	95.52	77.66	↑
	Hyperparameter tuning	no	90.66	76.17	↑
Base CNN	-	yes	79.61	79.67	↑
Deep CNN	-	yes	92.32	89.15	↑
ResNet 18	freezed convolution layer	no	90.25	87.83	↓
ResNet 18	Full train (2000 images)	yes	99.40	85.40	↑
ResNet 18	Full train (40,000 images)	Yes	99.50	93.81	↑

Starting with a simple CNN model



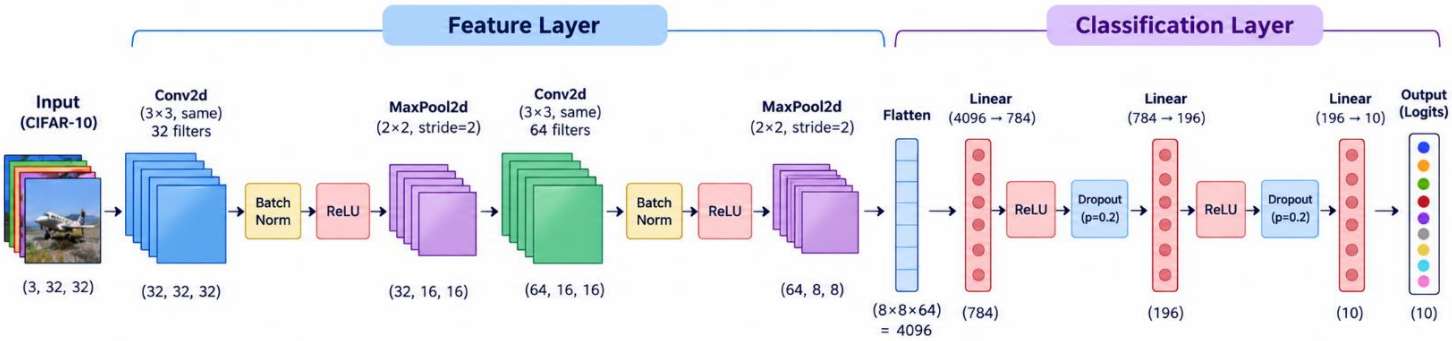
This is the base CNN architecture of 2 convolution layers (with pooling) in the feature network and 3 layers in the classification network.

First, training it on the 40,000 images for 40 epochs at a learning rate of 0.001, without early stopping gives a validation score of 68.83% and training score 98.82%. This was highly overfitting.

Then I introduced early stopping to reduce overfitting, and it gave similar performance with just 11 epochs. This saves a lot of time and compute resources, so early stopping will be used in every model trained after this.

Since deep learning models are highly prone to overfitting due to their complexity, to further reduce overfitting, I will incorporate some regularization technique techniques like batch normalization, dropout, weight decay.

Regularization to minimize overfitting



Similar architecture as before, but with batch normalization, dropout and weight decay. This is regularization and it helps the model to generalize better and reduce overfitting.

It gave 73.89% on validation set and 90.45% on train set.

That's a noticeable improvement. Overfitting is reduced considerably and the model is doing better on validation set. We will use regularization in every model trained after this.

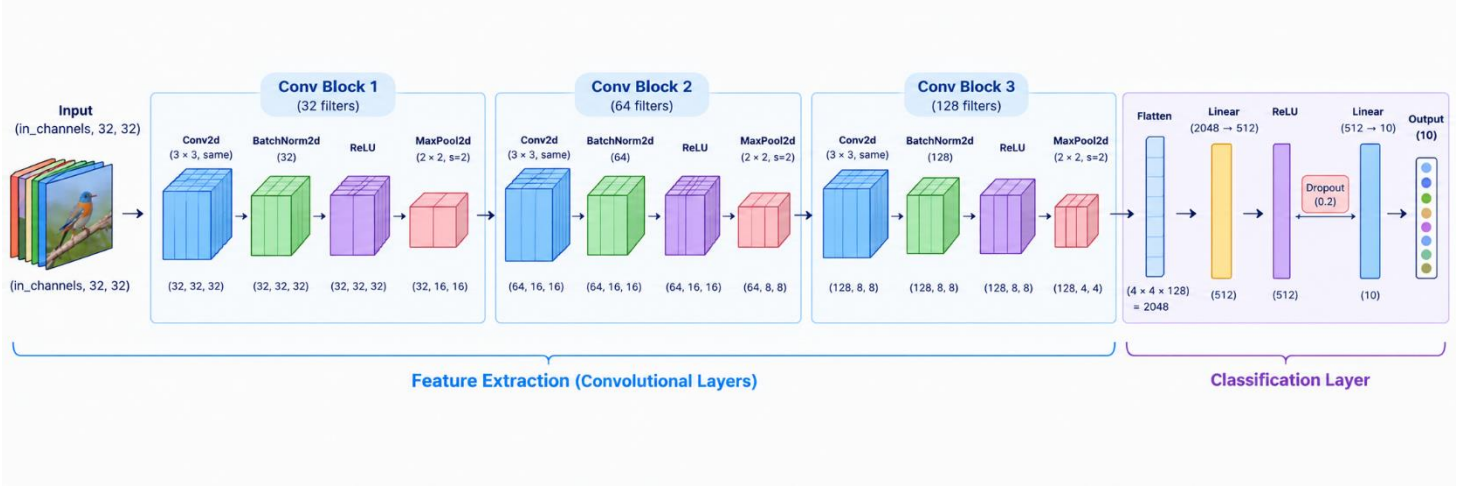
But there is still some room left to bring down overfitting.

At this point, my hypothesis is that:

- reducing the pooling layer might help here because the image size is already very small (32,32), and reducing it further by pooling will result in loss of information from an already low-resolution image.

- the classification layer shouldn't be too complex, because the convolutional layers have already extracted meaningful features from the images, reducing the complexity of the classification layer may perform better.

Modifying the architecture



I first tried removing the pooling layers, hoping that this will improve performance on low resolution 32x32 images. But this wasn't the case because, completely removing pooling layers resulted in performance drop, bringing down the validation score to 64.08%.

Next, I tried reducing the complexity of the classification layer and that performed slightly better with a validation score of 75.16%. It was also a bit faster to train.

Then I introduced one more convolution layer, and it gave a validation score of 77.66% and a training score of 95.52%. Overfitting is still there but this is by far the best score.

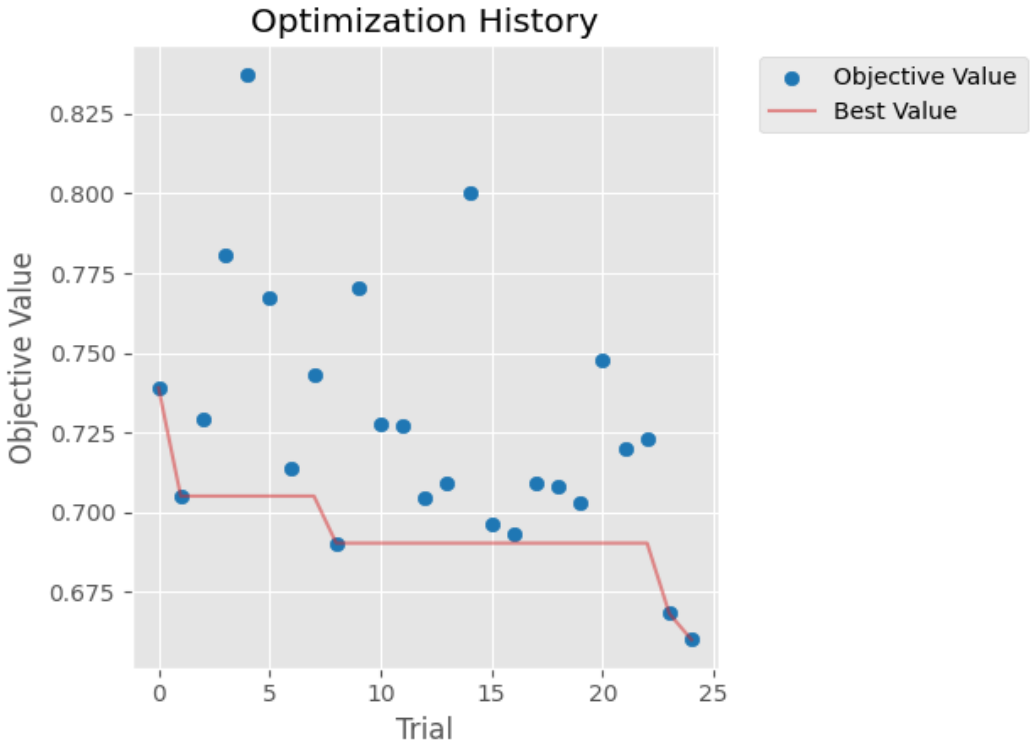
Hyperparameter tuning on learning rate, weight decay, dropout rate etc. can improve performance further.

Hypeparameter tuning

I used the Optuna library to find the best parameters for learning rate, weight decay, dropout rate and the number of neurons in a classification layer. Ran the optuna study for 25 trials on full training data.

It did bring down the overfitting but I wasn't impressed by the validation score which was 76.17% (lesser than before)

This plot visualizes loss trend with each trial:



It shows that optuna was going in the right direction to find the best parameters for minimum loss.

I find Optuna as definitely a useful library, but here, it was a resource intensive process with little improvement.

Data Augmentation and a Deep CNN model



Deep CNN

If the model is overfitting, then you may collect more training data to improve the overall performance and generalization, this is very much true for deep learning models.

But if more training data is not available, you can try data augmentation which works almost like a magic trick, especially in computer vision tasks.

Data Augmentation can improve the model performance by generating synthetic data by slight variations from our original dataset.

For example, take this image in our dataset.



Now, its intuitive to think that in real world, a model can get different variations of this horse image during prediction, like a different angle, brightness etc:

we can generate more images similar to this, by doing some variations like flipping, cropping, rotating slightly, adding noise to change brightness, saturation etc.

Look at these generated images below:



Luckily, we have inbuilt functions for this type of image transformations in Pytorch. Also, these images are generated on-the-fly, means they are generated only when requested during training so it does not consume more space, but on the other hand, it will take more time during training since the images are to be transformed when requested.

Base CNN + Data Augmentation:

First, I used the Base CNN model and it gave a score of around **79%** on both training and validation set. Because the model was trained on a lot of random variations of our original data, the overfitting has drastically reduced. In fact, "*There is NO overfitting at all*".

At this point:

- I want to increase the complexity of my model because now we have more diverse data and we need more complex model to capture these diverse patterns in our data
- I also observed more fluctuations in the validation loss due to random variations in the data for each epoch, so I should also increase the patience level of early stopping when training my next model (Deep CNN).

Deep CNN + Data Augmentation:

This CNN model had 3 blocks in its feature layer. Each block had 2 convolution layer and 1 pooling layer. The classification layer had 512 neurons. This model had relatively more parameters than the base CNN model. Now it could learn more complex patterns.

Training this model with data augmentation, gave **89.15%** accruacy on validation and **92.32%** accuracy on train set. That's a massive gain in performance, also the Overfitting is reasonably lower now.

Just by generating new images from variations in original image, we could push the accuracy by huge margin.

Next, we will try Transfer Learning,

Transfer Learning

Transfer Learning is a technique in Machine Learning, where typically a model trained on huge amount of data for solving one task is used to solve a different but related task. The pre-trained model is fine-tuned on task-specific dataset to achieve this.

Transfer learning is very useful when our task requires a lot of training data and compute resources, but we want to achieve good results with a small dataset available.

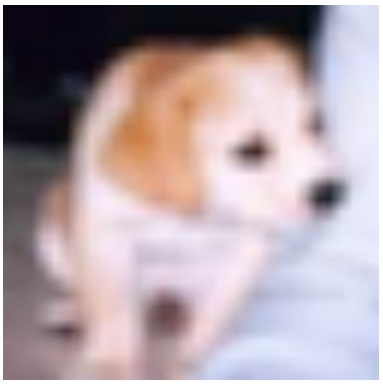
We will try **ResNet-18 model**, which is 18-layer deep convolutional neural network model. It uses residual (skip) connections to prevent the vanishing gradient problem during training. It is pretrained on huge ImageNet dataset and contains approximately **11.7 million parameters**.

Training ResNet-18:

In the context of ResNet-18, when we say its a pre-trained model, it means it already has the learned weight values which is the result of its training on huge data. The ImageNet dataset had 1,000 classes. We will just replace the output classifier layer and train this model on our dataset to predict on 10 classes. Also, this model was trained on images of size 224x224, so we will resize our 32x32 images before feeding into model. This resizing is also handled easily by pytorch's transformation pipeline.



32x32



224x224

Any image is made up of pixels that make edges, then edges make basic shapes and further complex shapes. The idea is that the feature layer has already learnt to extract useful features of an image, like edges, shape, color etc. and we will get these features as output from feature layer and use it to train our classifier layer to predict from 10 classes.

First, I trained this model by freezing the feature layer weights and training only the classifier layer. The result was **87.83%** accuracy on validation set (not better than our Deep CNN model).

ResNet-18 + Data Augmentation:

Then I trained the full model (where pretrained weights were also allowed to update) with data augmentation. When I started training, I was getting huge loss.

Then I realized that: Because ResNet18 backbone already has learned weights from ImageNet, with a learning rate of 0.001, those useful weights were updating too aggressively, causing the loss to start high or become unstable. Those weights are already around some optimum level; we just need to fine-tune them on our data. On the other hand, the classifier layer weights were randomly initialized, so it required more learning rate than feature layers. Hence, I used different learning rate for the feature layer (1e-6) and classifier layer (1e-4).

Then Training it on 40,000 available images, gives a validation score of **93.81%** and training score of **99.59%,** And this becomes our **best score till now.**

It was indeed a big model and with data augmentation it took a lot of time to train. Even with the T4 GPUs available on Kaggle, it took more than 2 hours.

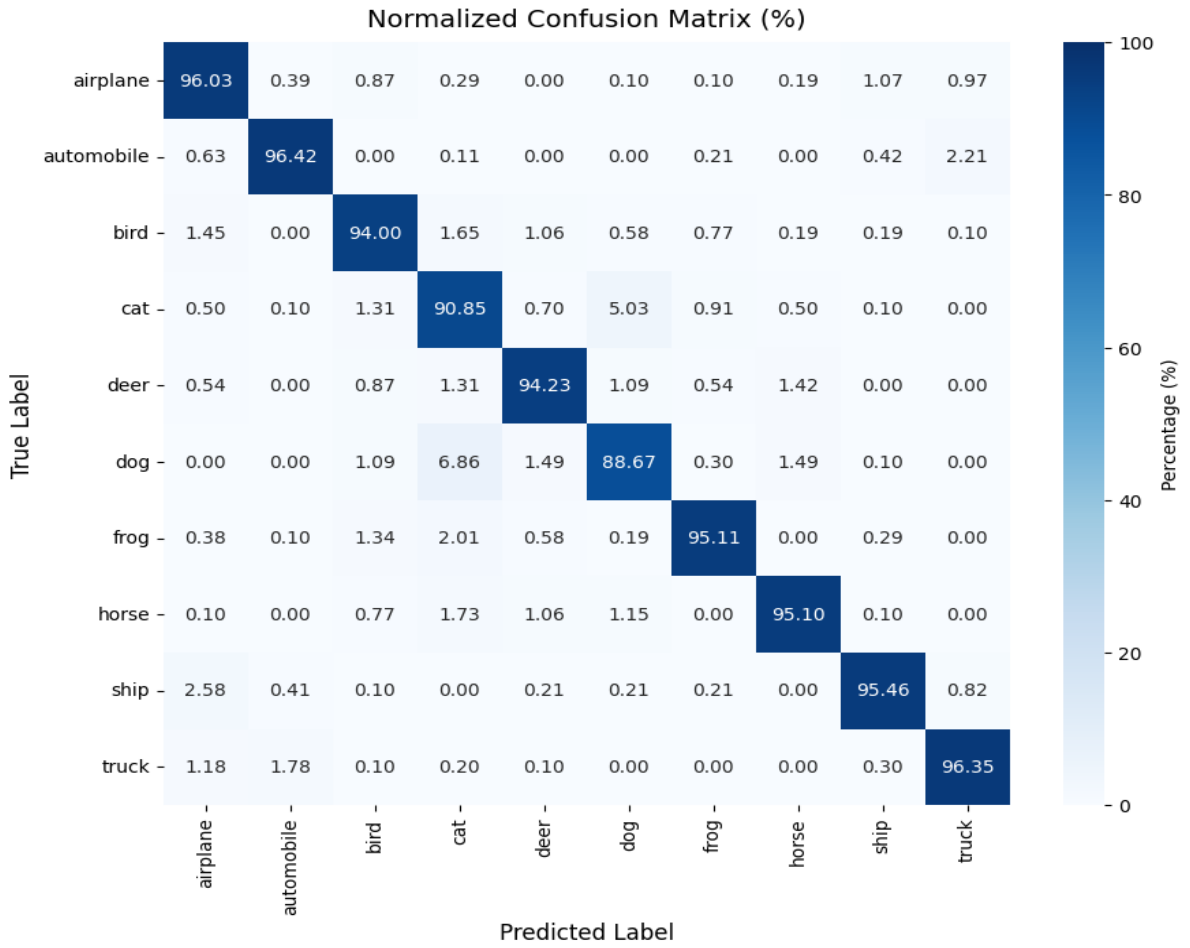
Now comes the most interesting part:

When training it on small random sample of 2000 images, this model gives a validation score of **85.44%**.

With Just 4% of our training data, this result is no doubt absolutely great compared to what we have been trying to achieve without transfer learning and with our full training data. This suggests that the pretrained features of ResNet18 model were transferring well to CIFAR-10 data.

This is where Transfer learning can be very useful, when we don't have enough data required for training a deep learning model.

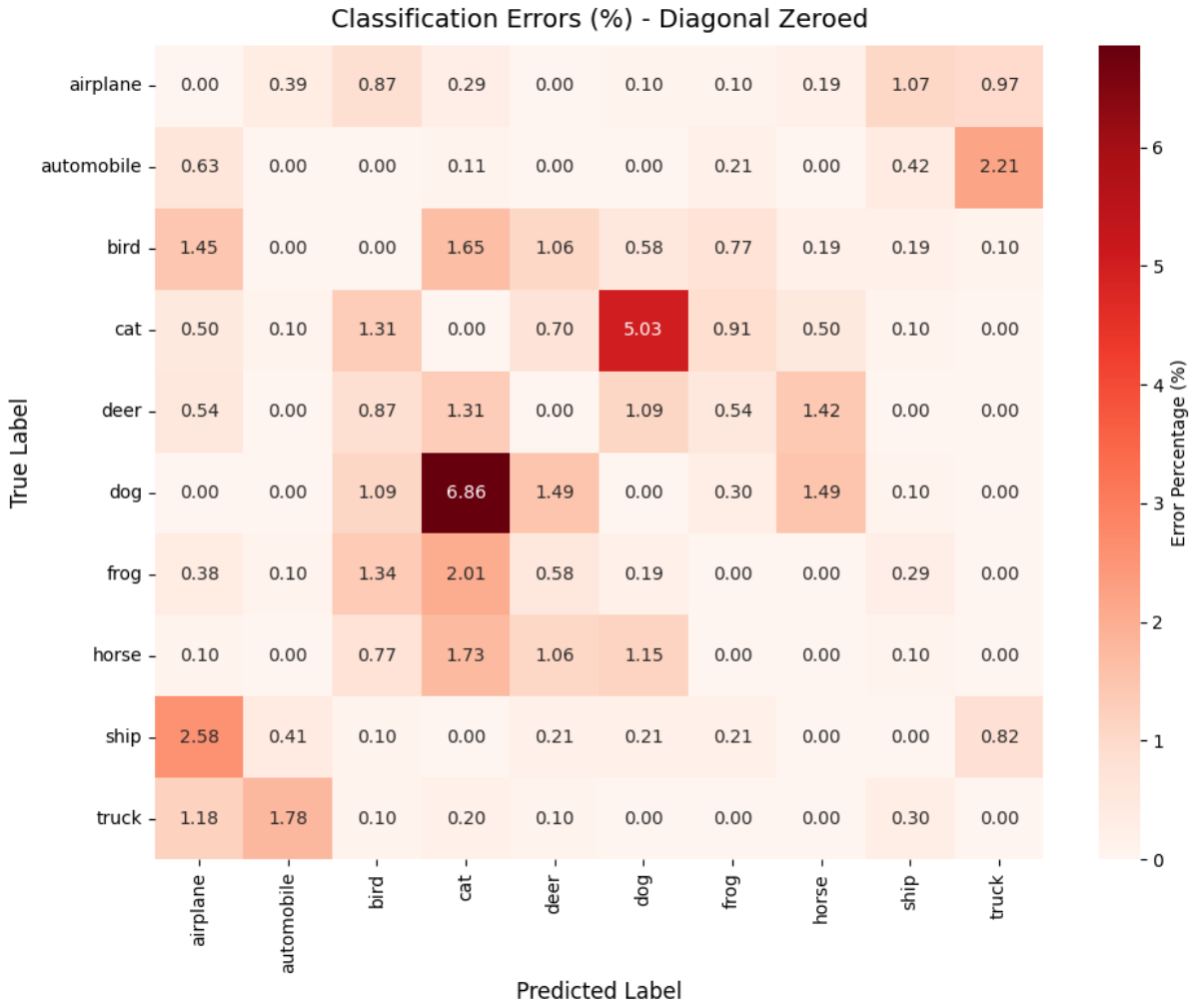
Error Analysis



This heatmap shows the performance of our best model across different classes. The diagonal values are basically the recall % for each of the class.

For example, a true airplane image is correctly classified as an airplane 96% of the time. A true bird image is classified as bird 94% of the time and etc.

All the off-diagonal values are basically error %, for example a bird gets misclassified as an airplane for 1.45% of the time. For clearly visaulising errors, let's set all the diagonal values to zero.



This gives much better view of the errors.

Some obvious error we can see here:

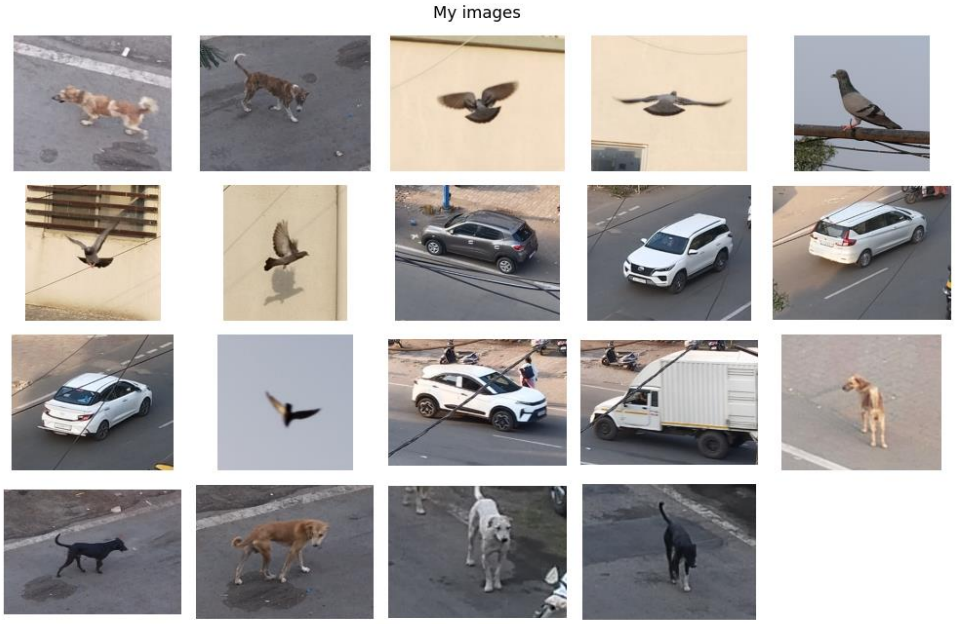
- automobilies misclassified as truck
- ships misscalassified as airplane

but **most importantly, the category dog and cat**. The error on these two is very high compared to other categories.

The model is finding it difficult to differentiate between dog and cat. This should be intuitive that cats and dogs do share a lot of visual features when looked at a distance, and, the fine details vanish in tiny 32x32 CIFAR10 images.

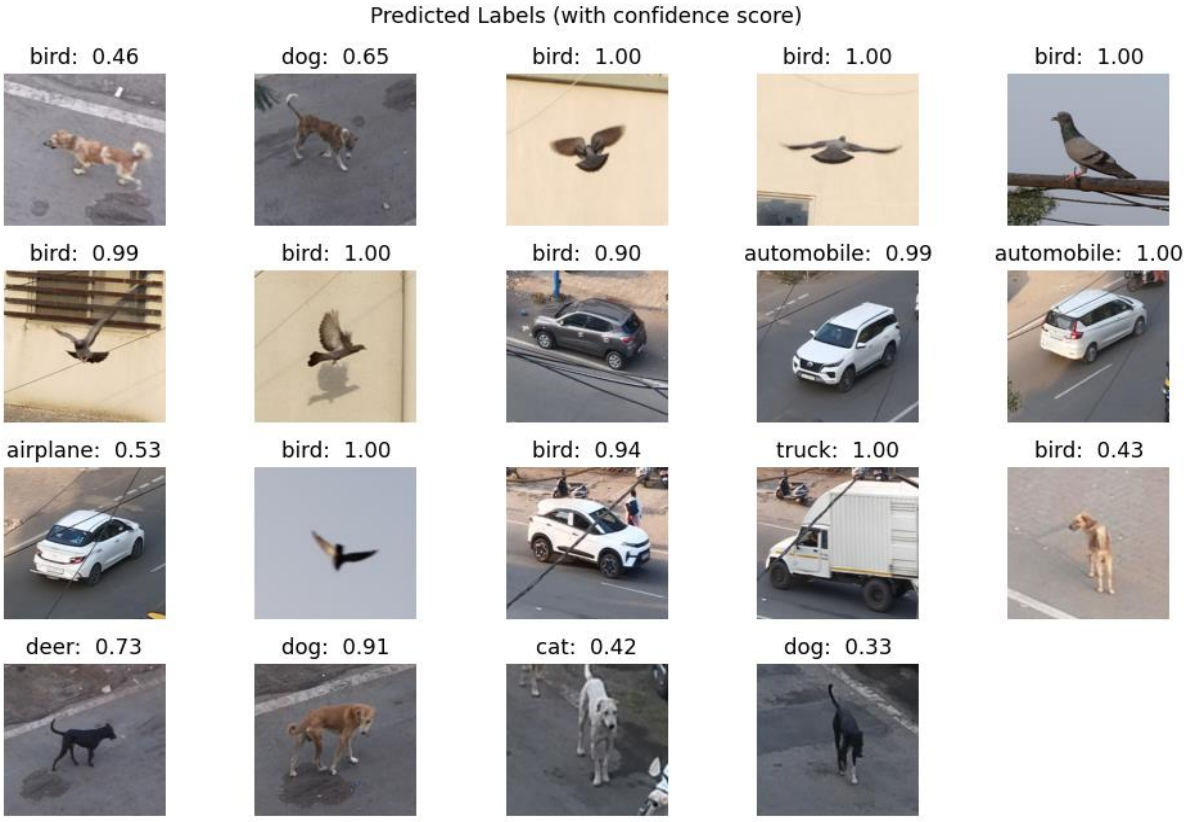
It's clear that these two categories have highest error and these two alone hurts the model's overall performance significatly.

Generating prediction for my images



Note: These are some pictures that I could manage to collect around my neighborhood.

(not very close to the type of data that the model was trained on, since the model was trained on close-range pictures, but I captured these images from an apartment, pointing my camera downwards at distance objects on the ground)



One drawback of CNNs or any neural network models is their explainability. For example, here we cannot know exactly why that 3rd image in the second row was classified as bird with such high confidence.

Acknowledgement

All the experimentations carried out in this project was possible because of cloud based GPU units (limited free) provided by Kaggle and Google Colab

Also, thanks to the Pytorch community for developing such useful opensource framework for deep learning.

END